COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 08, 2026	**DATE SUBMITTED:**	September 08, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort

Code

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sorting.h"

const int size = 100;

int main (){

    int arr[size];

    srand(time(0));

    for(int i = 0; i< size; i++){
        arr[i] = rand() % 5+1;
    }

    std::cout<<"Array: "<<std::endl;
    displayArray(arr, size);
```

```cpp
template<typename T>
void displayArray(T arr[], int size){
    for(int i = 0; i < size; i++){
        std::cout<<arr[i]<<" ";
    }
}
```

Output

```
Array:
2 2 2 3 3 1 4 2 5 1 3 3 5 1 3 4 5 2 1 3 3 4 3 1 2 4 1 4 3 1 1 2 5 5 3 2 5 1 1 3 4 1 4 1 4 2 4 4 3
 2 1 2 4 1 4 3 2 3 3 4 3 4 2 4 1 2 5 1 5 2 3 3 1 2 2 4 1 3 2 4 5 1 5 4 5 5 1 5 2 2 2 2 3 5 1 3 3
2 1 4
```

Observation

The code utilizes cstdlib and time.h to generate random numbers using srand(time(0)) which determines the start of the number and the time. I then implemented a for loop to go through each of the indexes in the 100 size array. For each iteration, an index is set to a random number by using rand() % 5+1, which pretty much generates a random number from 0 to 5. I then created a simple for loop that displays each element in the array.

Table 7-1. Array of Values for Sort Algorithm Testing

Code

Sorting.h

```
template <typename T>
void bubbleSort(T arr[], int maxSize){
    for(int i = 0; i< maxSize; i++){
        for(int j = i+1; j < maxSize; j++){
            if(arr[i] > arr[j]){
                std::swap(arr[i], arr[j]);
            }
        }
    }
}
```

Main

```
std::cout<<"Bubble Sort: \n";
bubbleSort(arr, size);
displayArray(arr,size);
```

Output

```
Bubble Sort:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3
 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5
5 5 5
```

Observation
The code works by setting a for loop and another for loop inside of the first for loop. It sets the first for loop, i, to the first index while the second, j, to one ahead of i. It then compares i and j, if i is greater than j, it is swapped. This will continue until the array is sorted or the program doesn't have any more values to compare.The number of comparisons made in a bubble sort can vary. However, since there are two for loops, we can get the number of comparisons by using n(n-1)/2. In this case it will be 100(100-1)/2 which is about 4,950 comparisons in the worst case. The time complexity for a bubble sort is O(n^2) for both best and worst case since it uses two for loops.

Table 7-2. Bubble Sort Technique

Code

Sorting.h

```
template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int position = k;

    T smallestElement = arr[k];

    for(int j = k+1; j < arrSize; j++){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}
```

```
template <typename T>
void selectionSort(T arr[], int arrSize){
    int min;

    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }
}
```

Main

```
std::cout<<"Selection sort: \n";
selectionSort(arr, size);
displayArray(arr,size);
```

Output

```
Selection sort:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3
 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5
5 5 5
```

Observation

The code uses the routine_smallest() function to find the smallest element in the unsorted array. It first assumes that the element at the current index is the smallest, if it is, it stores its value in the smallestElement variable. A for loop is then used to go through each after it and checks if there is a smaller value, if it does find it, the variable is updated and saved to the position variable.When the smallest element is found, the selectionSort() function uses swap to switch it with the element at the current index. The process will repeat until all of the elements are arranged from smallest to largest. To get the total comparison, we can also use the same formula, which makes the total comparison at the worst case to about 4,950 comparisons. Time complexity for this method is O(n^2) for both the best and worst case. This is because, like the bubble sort, it uses two for loops, one to find the smallest number and one for the main selection sort function.

Table 7-3. Selection Sort Algorithm

Code

Sorting.h

```
template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j = 0;
    T temp;

    while(k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
        k++;
    }
}
```

Main

```
std::cout<<"Insertion sort: \n";
insertionSort(arr, size);
displayArray(arr,size);
```

Output

```
Insertion sort:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3
 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5
5 5 5
```

Observation

The code works by taking one element at a time and then placing it into the correct position in the already sorted part of the array. The k variable is set to 1 since we are assuming that the array is already sorted. Temp stores the current element that needs to be inserted, while the variable j starts at the index before k so it can compare the current element with the element before it. The while loop checks if temp is smaller than the previous

element and the previous element is moved to one position to the right if it is. Once the correct position is found, temp is placed into the position. This continues by increasing k until all of the elements are sorted. If the array is already sorted, the total comparisons can equal to 99 comparisons while in the worst case, we use the same formula, getting 4,950 comparisons. The time complexity for this is O(n) if the array is already sorted and O(n^2) since it uses two while loops in the worst case.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Pseudocode of Algorithm

```
                                MAIN.CPP
START
      SET size = 100
      DECLARE array
      INITIALIZE random numbers

      FOR i=0 to size - 1
            Assign arr[i[ to a random number from 1 to 5
      END FOR loop

      PRINT "Original List of Votes"
      CALL displayArray function

      PRINT "Sorted List"
      CALL bubbleSort function
      CALL displayArray function

      PRINT "Candidate 1:"
      CALL countVotes function
      PRINT value of count

      PRINT "Candidate 2:"
      CALL countVotes function
      PRINT value of count

      PRINT "Candidate 3:"
      CALL countVotes function
      PRINT value of count

      PRINT "Candidate 4:"
      CALL countVotes function
      PRINT value of count

      PRINT "Candidate 5:"
      CALL countVotes function
      PRINT value of count

END
```

```
                        displayArray() function
FUNCTION displayArray(arr, size)

      FOR i=0 to size - 1
            PRINT the current index of the array
      END FOR

END FUNCTION

                        bubbleSort() function
FUNCTION bubbleSort(arr, size)

      FOR i=0 to size - 1
            FOR j = i + 1 to size - 1
                  IF arr[i] is less than arr[j] THEN
                        SWAP arr[i[ and arr[j]
                  END IF
            END FOR
      END FOR

END FUNCTION

                        countVotes() function
FUNCTION countVotes(arr, size, candidate_number)
      SET count = 0

      FOR i = 0 to size - 1
            IF arr[i] is equal to candidate_number THEN
                  INCREASE count by 1
            END IF
      END FOR

      RETURN count

END FUNCTION
```

Screenshot of Algorithm Code

```
#ifndef VOTE_H
#define VOTE_H

#include <iostream>

template<typename T>
void displayArray(T arr[], int size){
    for(int i = 0; i < size; i++){
        std::cout<<arr[i]<<" ";
    }
}

template<typename T>
void bubbleSort(T arr[], int size){
    for(int i = 0; i < size; i++){
        for(int j = i+1; j < size; j++){
            if(arr[i] > arr[j]){
                std::swap(arr[i], arr[j]);
            }
        }
    }
}

template <typename T>
int countVotes(T arr[], int size, int candidate){
    int count = 0;
    for(int i = 0; i< size; i++){
        if(arr[i] == candidate){
            count++;
        }
    }
    return count;
}

#endif
```

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "vote.h"

const int size = 100;

int main(){
    int arr[size];

    srand(time(0));

    for(int i = 0; i < size; i++){
        arr[i] = rand() % 5 + 1;
    }

    std::cout<<"Original List of Votes: ";
    displayArray(arr, size);

    std::cout<<std::endl;

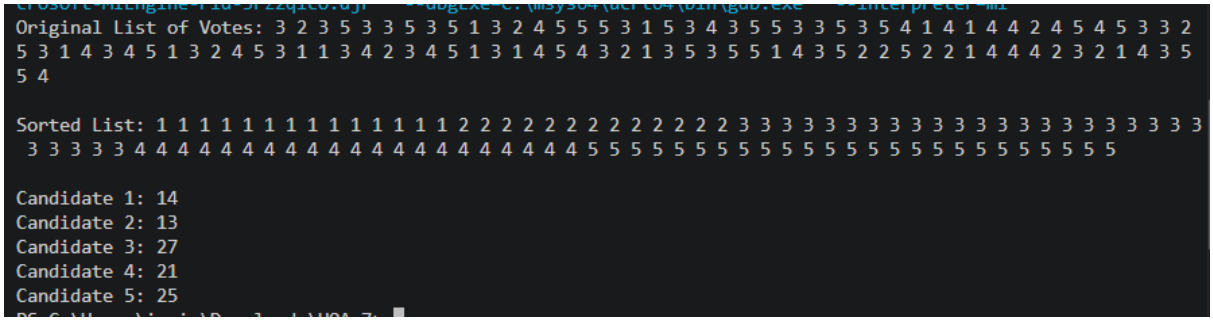
    std::cout<<"\nSorted List: ";
    bubbleSort(arr, size);
    displayArray(arr, size);

    std::cout<<std::endl;

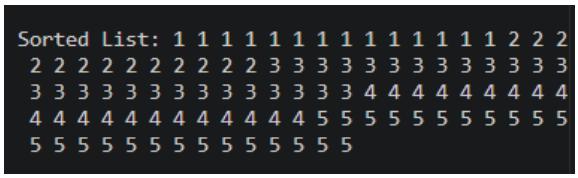
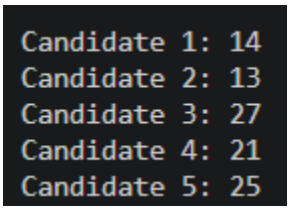
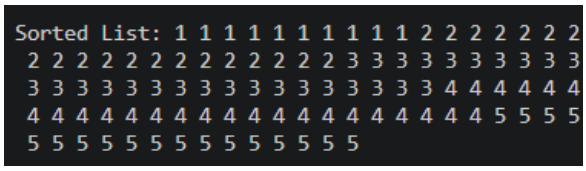
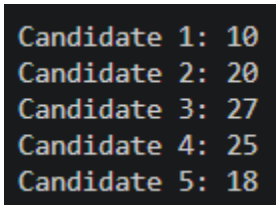
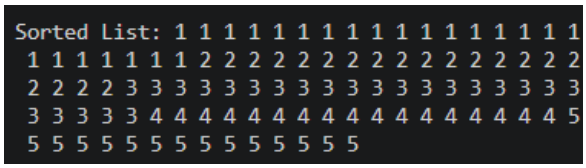
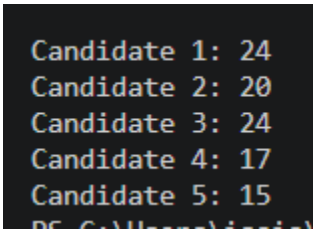
    std::cout<<"\nCandidate 1: ";
    std::cout<<countVotes(arr, size, 1)<<std::endl;
    std::cout<<"Candidate 2: ";
    std::cout<<countVotes(arr, size, 2)<<std::endl;
    std::cout<<"Candidate 3: ";
    std::cout<<countVotes(arr, size, 3)<<std::endl;
    std::cout<<"Candidate 4: ";
    std::cout<<countVotes(arr, size, 4)<<std::endl;
    std::cout<<"Candidate 5: ";
    std::cout<<countVotes(arr, size, 5)<<std::endl;

}
```

Output Testing



Why Bubble Sort: I chose bubble sort since it is the most simple and easiest way to sort data for me. It simply works by comparing adjacent elements and then swapping them when they are in the wrong order. Since the array only needs 100 elements and the values only goes up from 1 to 5, bubble sort is good for the activity.

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	**Candidate 1: 14** **Candidate 2: 13** **Candidate 3: 27** **Candidate 4: 21** **Candidate 5: 25**	
	**Candidate 1: 10** **Candidate 2: 20** **Candidate 3: 27** **Candidate 4: 25** **Candidate 5: 18**	
	**Candidate 1: 24** **Candidate 2: 20** **Candidate 3: 24** **Candidate 4: 17** **Candidate 5: 15**	

Question: Was your developed vote counting algorithm effective? Why or why not?

- I believe that the developed vote counting algorithm was effective since I was able to make it so that it sorts the 100 votes and correctly counts the votes for each candidate, even matching the values that were counted manually. The bubble sort also made it easier to implement the sorting system. However, I do believe that there are faster and more efficient ways to sort elements when the amount of data is very large. However, bubble sort was sufficient since the number of elements isn't too large.

Conclusion
In the activity, I was able to learn about how different sorting algorithms work. The lesson taught me about bubble sort, selection sort, and insertion sort. I was also able to apply bubble sort in the supplementary activity to sort the votes and count how many votes each candidate received. With the result, the algorithm I created was able to correctly count the votes and even matched the values that I counted manually. Furthermore, I learned that bubble sort is one of the simplest way to do sorting but it can still vary based on the size of the data that is being sorted. Overall, the activity helped me understand how sorting algorithms can be used and how important it is to have an organized system that makes it easier for the data to be interpreted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.